Summarize

Chapter 2. Important Distributed System Concepts

Before we get too deep into the design of distributed systems, it is valuable to ground ourselves in some key concepts that underpin the development of these distributed systems. If you are already experienced in distributed systems or systems design in general, some of these ideas may be familiar to you, and you may want to skip ahead, but hopefully at the completion of this chapter, you will have a core foundation for the patterns and designs presented throughout the rest of the book.

APIs and RPCs

Application programming interfaces, or APIs, form the core of any distributed system. You may have seen APIs in the context of programming language libraries or calls to operating system APIs. In distributed systems the idea is similar, but the API calls take place over the network between remote services. These calls are referred to as "RPCs" or "remote procedure calls." RPCs rely on an underlying network transport, commonly, though not exclusively, the HTTP protocol and the JSON object format. Numerous other more structured protocols have existed over the years, from CORBA to gRPC as well as database-specific protocols. Though such systems provide capabilities beyond HTTP + JSON (also referred to as REST or RESTful), these continue to be the most popular because of their wide support, simplicity, and ease of integration with web-based frontends.

APIs can be synchronous or asynchronous. Synchronous APIs return only after the request has been fully processed, while asynchronous APIs accept the request and return an operation ID for a background process that can be monitored via subsequent status requests. Synchronous APIs are the easiest to implement, but long-running operations are a bad match for HTTP or other network protocols because of timeouts and other limitations. Thus, if the length of your operation is measured in minutes, it is better off being modeled as an asynchronous operation.

APIs are generally not very useful without clients that can call those APIs. Having every developer hand-roll HTTP calls to your API is a recipe for unhappy developers and badly designed systems. In smaller systems, which may only be required to support individual languages, it is possible to handwrite these client libraries. But larger systems rely on IDLs, or interface description languages, such as OpenAPI, which can describe your API's structure in such a way that clients can be automatically generated for nearly any programming language.

Beyond the structure of your API, the core functionality of an API is described in terms of its SLO, its service level objectives, which describe how the API performs in terms of latency and reliability. Having formal SLOs for latency, reliability, and throughput (requests per second) enables clients to successfully develop distributed systems.

Latency

One of the most important measures of your system's performance is the time that it takes to do something. This measure is commonly referred to as *latency*. Generally speaking, latency is measured in milliseconds, though depending on your system, it could be measured in microseconds (very fast) or seconds (very slow). When measuring latency, it is important to ensure that it is actually representative of end-user experience (see "Percentiles") and that it is attributable, meaning you understand where the latency is coming from. Without proper attribution, you may understand that your system is slow, but you will have little idea about how to make it better.

Reliability

In addition to latency, reliability is obviously a key metric for any distributed system. At its simplest, reliability is just the fraction of successful requests relative to the total number of requests. But as with all things engineering, in the real world, the devil is in the details. Consider requests that are based on the HTTP(S) protocol. Using this protocol, you could measure the number of requests failed (i.e., requests that didn't actually complete the HTTP protocol). However, for any reasonable web service, this number rapidly trends to zero. The reason for this is that HTTP itself has rich error semantics in the form of the HTTP response code. Thus, nearly every HTTP request succeeds at the protocol level, but often the response code indicates an error. From this, it is clear that our measure of reliability should not be at the HTTP protocol level, but rather at the HTTP response code level. However, HTTP defines a large number of response codes. Which should we treat as errors? It's fairly obvious that the 50*x* series of codes are true

errors. Most of them represent either internal server errors (500) or network transmission errors (e.g., 502). Similarly it is fairly clear that the 20*x* series codes are successful requests. Unfortunately, in between 20*x* and 50*x* is a gray area. Typically we can consider the 30*x* (mostly redirects) status codes to be "successful," though an excessive number of redirects *can* be an error in some conditions. Most of the uncertainty lies in the 40*x* collection of errors. The 40*x* collection of status codes are the land of ambiguity. Is that 404 (not found) an indication that a client simply made a mistake? Or is your service somehow misconfigured to look for files in the wrong location? Similarly, it may seem obvious that 403 (unauthorized) represents a "client error," but what if your authorization service is having an outage?

Ultimately, though the examples above are drawn from the HTTP protocol, the important takeaway is that measuring reliability is both critical and complicated. In practice, treating 404 (not found) as a client error can mask true reliability problems and prevent you from alerting when there is a customer-facing outage in your system. At the same time, a spike in 404 errors can also be from a single misconfigured client, causing noise and false alerts, which will plague your on-call engineers. In this gray area, proper monitoring will be application dependent. Further AI-based operations and anomaly detection become critical in differentiating between true problems and false alerts.

Percentiles

Throughout this book there are discussions of percentiles. The 99th percentile latency, for example. Percentiles are key to understanding distributed systems because distributed systems are inherently about serving large numbers of users and large numbers of requests. Of course, in serving lots of users, we want to understand the operation of our services. By default it is easy to use the average as the way in which we measure user experience. It makes sense. We want to understand the experience of the average user, so it makes sense to look at the average experience of our various metrics. Unfortunately, averages are often really distorted in considering the experience of a system. There are several reasons for this, but the two largest have to do with the calculation of the average as well as the ways in which individual metrics (e.g., request latency) come together in an end-to-end user experience. When you consider the arithmetic average, it is heavily skewed by either large or small outliers. Consider measuring latency, a relatively small number of users with very fast latency will distort the average latency of your system and make you believe that your "average" performance is much better than it actually is.

The other reason that the average is problematic is that it considers individual requests separately; however, most user experiences are the combination of multiple requests

which must succeed. The combining of all of these experiences means that almost no one actually has an "average" experience in terms of latency; instead, the probability that every user has at least one exceptionally slow request rapidly approaches 100% as the number of requests in an end-to-end experience increases.

For both of these reasons, to truly understand the behavior of your system, it is far more useful to consider the 90th or 99th percentile of your user experience. These percentiles represent the place in the distribution of latency where 90% or 99% of your users experience *lower* latency. For example, if your 99th percentile latency is one second, then 99% of your users have latency less than one second. This gives you a much better sense for the operation of your service, but importantly, even the 99th percentile suffers from the problems of multiple requests in a single end-to-end experience. Taken together, the 99th percentile latency of a single request rapidly becomes the 90th (or even 50th) percentile of an end-to-end experience with enough requests in that experience. Because of this, it is always worth understanding exactly *which* experiences you are measuring.

Idempotency

Designing distributed systems is, in many ways, the study of, and anticipation of, how systems can fail. Of course, we don't want our systems to fail, and thus, in order to compensate for when the underlying systems *do* fail, we introduce ideas that make it easier for us to restore our services. The first of these is the idea of *idempotency*. Idempotency is the notion that whether an action is performed a single time or many times, the result is identical. An example of an idempotent operation is the statement X = 5. No matter how many times you execute that operation, the result is the same. X always is equal to 5. Compare that with the statement X = X + 1. This operation is *not* idempotent. Each time it's executed, the value of X increments by one. It is clear that the eventual value of X is dependent on the number of times that instruction is executed. If you are familiar with the ideas of infrastructure as code and declarative configuration, idempotency maps directly to declarative versus imperative configuration.

Idempotency is an important concept for distributed systems because it makes error and failure handling significantly easier. If we know that an operation is idempotent, in the face of failures, we can simply repeat it until it succeeds. No matter how many times it takes to get a successful outcome, we know that that outcome will always be the same. Idempotent operations make designing reliable systems easier.

Delivery Semantics

Though idempotency allows us to ignore the number of times a particular request has been made, sometimes it is necessary to actually control the number of times a particular request has been made in the presence of error conditions.

Generally, delivery of a message (or the success of a request) is divided into two categories. The first is "at least once" delivery. As its name suggests, at least once ensures that a message or a request will be delivered at least once to the recipient. The "at least" part is the important part, because with at least once delivery there are no guarantees about how many times a message will be received.

In contrast to "at least once" delivery, the alternative is "at most once" delivery, which ensures that a message can never be delivered more than once. As with at least once, the important part of this is the "at most" phrase, because with at most once delivery it is perfectly acceptable for a message to *never* be delivered. A good example of where you might want at most once delivery is the charging of a credit card. Clearly multiple identical charges to the same card are not allowable; "at most once" delivery assumes that in the presence of failure (e.g., the absence of payment) someone will try again.

You may notice that "exactly once" delivery is missing from the options described. This is because, in practice, exactly once delivery is exceptionally hard to achieve. You are far better designing systems which can handle either at most or at least once delivery.

Relational Integrity

The reason that message delivery semantics become so important is that they ultimately affect the data that is the core of our application. If the data is your bank account, the integrity (e.g., reliability) of that data is clearly a very important concern. Ensuring the reliability of a single data entry (e.g., the balance in your bank account) is important, but *relational integrity* is an equally important concern for distributed systems. Relational integrity refers to the integrity or accuracy of data stored in multiple different rows or stores within a system. For example, your bank balance might be stored as a pair of `(user-id, balance)`, and your address might be stored in a different store as `(user-id, address)`. The integrity of that data refers to whether there are strong guarantees that for every `user-id` in the balances table, there is a corresponding entry in the addresses table. If the code in your application relies on there being consistency between various data stores, then your code relies on strong relational integrity. If instead your code can

handle cases in which data in multiple stores may not be fully in sync, your code does not rely on relational integrity. The reason that this becomes important is that maintaining relational integrity requires carefully synchronizing changes to multiple data stores. This synchronization has significant performance and reliability implications. In particular, it requires distributed transactions across multiple different data stores, which requires distributed locks that make your system more complicated and less parallel. Relational integrity can generally be seen as the distinction between SQL and NoSQL data stores. Traditional databases and SQL go to great pains to ensure relational consistency between various tables, including transactions and rollback semantics. NoSQL data stores like Azure's CosmosDB and the open source Cassandra database favor more relaxed relational semantics, where everything is a key/value store and consistency is a problem for application code. In general, relational consistency can be seen as a tradeoff between performance and code complexity. If you don't need performance, the consistency offered by a traditional SQL database can make your code much easier, while looser approaches facilitate increased parallelism and corresponding performance.

Data Consistency

Beyond relational consistency, there is another related, important dimension of consistency to consider, and that is data consistency. In any distributed system, data is replicated to ensure the availability of data in the face of errors. This data replication has performance implications, and it is important to decide whether your data requires *strong* or *eventual* consistency.

Clearly, consistency, the notion that the data written is the same in all locations, is a critically important part of data storage. But how quickly must that data be the same? That is what we are considering when we talk about data consistency. In a strongly consistent system, the data is guaranteed to be present everywhere before the writing of the data is considered to be "complete." In an eventually consistent system, the data is guaranteed to be the same everywhere "eventually," where eventually can actually mean a long-ish time.

To make this a little more concrete, consider a system which replicates its data to all of the data centers it maintains in the world. In a strongly consistent system, a user requests a write of some data item, and that data is immediately written to all of the different data centers. Until all of these data centers successfully acknowledge that the write has completed, the user request to write is not considered successful. At first blush, this seems like the right approach—until the data is everywhere, the user should not be told that their request is successful. Unfortunately, the result of this is signifi-

cantly decreased performance. As the number of data centers grows, the number of required writes also grows, up until a point where every request is basically guaranteed to hit a data center which is running slowly or even failing. Thus, while strong consistency ensures data reliability, it significantly reduces write throughput.

In the previous example, an eventually consistent system might return as soon as a single data center has acknowledged the write. Replication to all of the data centers is performed by an asynchronous background process, and the user doesn't have to wait around for it to complete. The system says "I got this," and the user trusts the system to eventually make the data consistent. Waiting only for a single write makes the system both more reliable and more performant. You are significantly less likely to hit a slow or failing data center with a single write.

Unfortunately, in many use cases eventual consistency gives the impression to the end user that the system is *less* reliable. The reason for this is the "read your own writes" problem. Distributed systems replicate themselves across multiple data centers for reliability and performance. Most of the time, a single user's requests will all go to the same data center, but occasionally, under load, failures, or other specific network conditions, different requests from the same user will go to different data centers. In an eventually consistent system, this means that after a user writes something to the system, a subsequent read that is load balanced to a different data center may not see the recent write. This can be quite distressing to a user. Imagine if you ordered something on a retail website, clicked on "view my orders," and couldn't see your order in the returned list. It's highly likely that you would never order from that site again.

Because of these complications, the choice between strong consistency and eventual consistency is never a very clean one. It depends on the use case as well as the performance requirements of the system. Be careful when deciding which approach to take with your system, though. The decision to use strong or eventual consistency will permeate your system design and your application's code. Once written, it is very hard to change the storage model that is used by a distributed system, and the implications of the decision often take years to show up.

The trade-offs above between *consistency*, *availability*, and *partition tolerance* is known as the CAP theorem, and it states that you can build a system with two of the three attributes, e.g., a system that is consistent and available but not partition tolerant, but you can't build a system with all three. It's worth noting that the CAP theorem is not a draconian rule. You may not be able to build a system that guarantees all three, but it is possible to build a system where each are reasonable for your application. For some applications, 99% availability is sufficient, and thus, you can use that trade-off to ensure the

consistency and partition tolerance you need as well. Effectively, the CAP theorem says that whenever you are optimizing one of the system characteristics, you are inherently trading off against other goals.

Orchestration and Kubernetes

Distributed systems defined today are not deployed in a vacuum or using only code written by the system's developers. Instead, the ubiquity of distributed systems today has meant that shared infrastructure has been built to support the deployment and operation of distributed systems. This infrastructure is commonly referred to as an orchestrator, because it orchestrates the operation of the distributed system without implementing the details of the systems logic. The most common orchestrator is the open source Kubernetes system for orchestrating deployments of containerized applications. The orchestrator is responsible for taking the desired state of the application (e.g., I want three replicas of my frontend and five replicas of my backend) and ensuring that the real state of the system matches the desired state that has been declared.

Health Checks

To achieve successful orchestration, the orchestration system needs to understand the health of the pieces that make up the distributed system. Health is used to understand when an application is ready to be added to a load balancer, when an application is unhealthy and needs to be restarted, or when a rollout should be halted because the health of the application is impacted. While there are some basic aspects of health (e.g., is the program running) that the orchestrator can observe from outside the system, true understanding of your application requires that you implement health checks that are specific to your application. When providing health information to the orchestrator, there are two different notions of application health. The first is the simplest to understand, in that it simply represents whether or not your application is alive. Applications which are no longer responding to a *liveness* check should be terminated. And if a rollout is causing a significant degradation in the number of alive applications, that rollout should be halted. The second notion of health is *readiness*. Readiness indicates that an application is ready to be used. At first blush, readiness and liveness would seem to be the same thing, but in many circumstances an application can be alive but not ready. The easiest example to understand is when an application has just started up and is initializing its state, perhaps by downloading and caching large files over the network. Such an application is alive, it is doing useful work and should not be terminated, but it is not ready until it has successfully downloaded and made ready to serve all of the files in its cache. By

properly implementing both liveness and readiness checks, you can ensure that your distributed systems interact well with the orchestrator on which they run.

Summary

This chapter has described a number of key concepts that are foundational to the design and understanding of distributed systems. These concepts form the basis for both understanding the requirements and describing the characteristics of the systems that we build. Before going further into the book, make sure that they all feel intuitive and familiar to you.

Previous chapter
< 1. Introduction

Next chapter
II. Single-Node Patterns >